TRACES Open Source Tracking

Purpose

This document tracks the public development direction for TRACES. It is intended for users, researchers, and developers who want to contribute improvements, compare versions, or understand which parts of the application are ready for extension.

TRACES is a modular PyQt6 desktop application for importing, visualizing, and analyzing vehicle trajectory data. The current codebase already has clear extension points for new import formats, analysis modules, plot interactions, and export workflows. This document highlights those areas and leaves room for contributors to propose recommended implementations.

Version Tracking

Version	Date	Status	Main Changes	Notes
1.0.0	TBD	Released	JSON-configured CSV import with unit conversion; SQLite trajectory storage; interactive time-space plotting, event/lane filtering, vehicle search, and speed coloring; gap/headway measurement and semi-automatic shockwave fitting; spline fitting, jerk, missing-data, internal-consistency, platoon-consistency, and spectral analyses; leader/follower and lane-change statistics; stacked kinematic plots; gap interpolation, lane-anomaly correction, and trajectory stitching with provenance and undo; CSV and interactive HTML export; Windows installer and portable builds	Current development baseline
1.0.1	TBD	Patch candidates	Bug fixes, documentation improvements, UI polish, targeted tests	Recommended for small first-time contributions.

When a release is prepared, add a new row with:

- **Version**: semantic version tag, for example `0.1.1` or `0.2.0`.
- **Date**: release date or planned date.
- **Status**: `Current`, `Released`, `Proposed`, or `Deprecated`.
- **Main Changes**: short user-facing summary.
- **Notes**: migration notes, known limitations, or links to related pull requests.

Current Capabilities

Area	Current Implementation	Extension Surface
Data import	JSON-configured CSV import with column mapping, transformation functions, unit conversion, reserved-column checks, and event summaries through `src/data_import/`; bundled profiles in `src/config/`	Add `BaseProcess` subclasses for other formats, CSV transformation functions, or dataset-specific profiles.
Data storage and provenance	SQLite databases with `events` and `ranges`, database validation, derived analysis columns, row and trajectory status codes, module-specific side tables, and saved superseded observations; write changes to the open database or a copy	Extend validation and schema migration support; register new derived columns and status codes, and use the shared provenance helpers in `src/analysis/base_comp.py`.

Area	Current Implementation	Extension Surface
Plotting and exploration	Interactive time-space plot with event/lane controls, time and distance sliders, speed coloring, zoom, hover details, vehicle search, and individual or group trajectory selection; stacked position, speed, acceleration, and jerk plots and statistical box plots	Add plot layers, annotations, or saved view states through `src/plot/` and reusable plotting widgets in `src/ui_kit/`.
Analysis modules	Jerk, missing-data, internal-consistency, platoon-consistency, spectral, B-spline fitting, and semi-automatic shockwave analyses	Add modules under `src/analysis/` using the shared computation, dialog, and worker classes; extend fitting methods, analysis parameters, or diagnostics.
Advanced statistics	Leader/follower and lane-change statistics for selected trajectories, with result tables, box plots, and CSV export	Extend statistic definitions in `src/analysis/advanced_statistics/stat_registry.py` and the associated computation and plotting workflows.
Data correction	Gap detection and spline interpolation with review and manual guide-point refinement; proximity and rapid-lane-change anomaly detection with lane reassignment; trajectory stitching with safety checks; provenance tracking and undo	Extend `src/analysis/gap_interpolation/`, `src/analysis/lane_anomaly/`, or `src/analysis/manual_stitching/` with additional detection or correction methods that preserve original observations and support undo.
Interaction modes	Trajectory selection, gap/headway measurement, semi-automatic shockwave point chaining and fitting with editable results, and trajectory-segment selection for stitching	Add manual and automatic shockwave handlers or other measurement tools through `InteractionModeHandler` in `src/plot/handlers/`; manual and automatic shockwave modes are not yet implemented.
Export	CSV export of complete datasets, selected event columns, and analysis results; self-contained interactive HTML export of analysis plots	Add JSON/Parquet writers, report bundles, or reproducible analysis manifests through `src/gui/export_dialog.py`, analysis result dialogs, and `src/ui_kit/plot_export.py`.

Recommended Implementations

The items below are intentionally written as contribution invitations. If you implement one, update this table with design notes, test coverage, and any limitations discovered during review.

ID	Recommendation	Why It Matters	Suggested Starting Point	Contributor Notes
RI-001	Add import support for additional trajectory formats	Many trajectory datasets are not delivered in the current CSV layout	`src/data_import/base_process.py`, `src/data_import/csv_process.py`, `src/config/`	Contributors can propose target formats here.

ID	Recommendation	Why It Matters	Suggested Starting Point	Contributor Notes
RI-002	Support multiple configurable spline-fitting methods	Different trajectory datasets and analysis tasks may require different spline methods and parameter settings	`src/analysis/spline/spline_comp.py`, `src/analysis/spline/spline_testing_dialog.py`, `src/analysis/spline/spline_comp_dialog.py`	Extend the current B-spline approach with additional selectable spline-fitting methods and method-specific parameters. Use a common fitting interface, retain the current method as the default, and support previewing and comparing fits before applying them. Test parameter validation and consistent position, speed, acceleration, and jerk outputs; document each method's assumptions and limitations.
RI-003	Add manual and automatic shockwave modes	Users need direct control over shockwave point selection and automated detection across trajectories	`src/plot/handlers/shockwave_handler.py`, `src/plot/handlers/interaction_mode.py`, `src/analysis/shockwave/`	Build on the existing semi-automatic mode. Support manual point selection and configurable automatic detection, with reviewable fits, wave speed, and fit-quality results. Document detection assumptions and limitations.

Add new recommendations below this line:

ID	Recommendation	Why It Matters	Suggested Starting Point	Contributor Notes
RI-004	*Open slot*	*What user or developer problem does this solve?*	*Relevant files or modules*	*Proposed by / status / links*

Contribution Workflow

Recommended workflow for contributors:

1. Read the User Guide to understand the workflows affected by your change.
2. Read the Developer Guide for the relevant package pattern.
3. Open an issue or discussion describing the proposed implementation, expected behavior, and any dataset assumptions.
4. Keep the first pull request small and testable.
5. Add or update documentation when the user-facing behavior changes.
6. Add tests when the change affects parsing, database writes, numerical results, plot state, or interaction-mode state.
7. Update the version tracking table or recommended implementation table when the change lands.

Implementation Guidelines

Prefer changes that match the existing architecture:

- Put import logic behind a `BaseProcess` subclass or a new CSV algorithm function.
- Put reusable computation in `src/analysis/`, not directly in GUI code.
- Put interactive plot behavior behind an `InteractionModeHandler`.
- Keep database assumptions explicit by checking available `events` columns before reading optional fields.
- Use dialogs and result tables consistently with the existing analysis modules.
- Avoid broad rewrites unless the proposal includes a migration path and test plan.